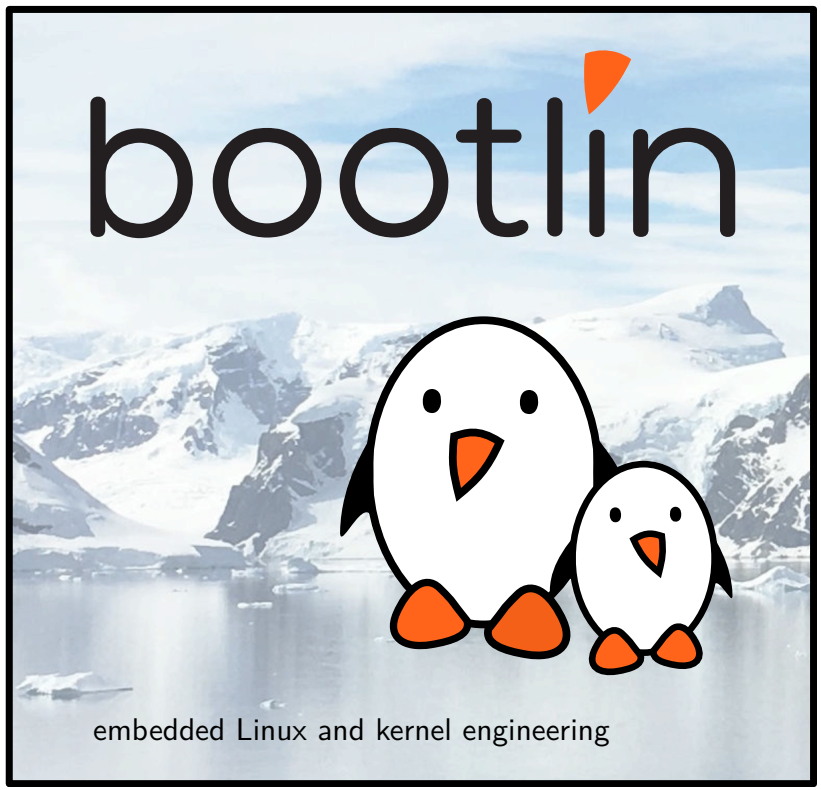
Yocto Project and OpenEmbedded system development training

© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Latest update: November 08, 2026.

Document updates and training details:
https://bootlin.com/training/yocto

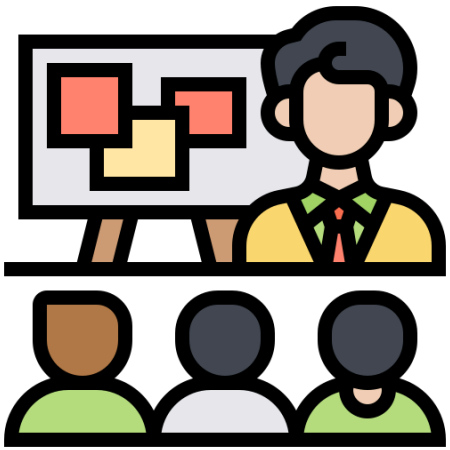
Corrections, suggestions, contributions and translations are welcome!
Send them to feedback@bootlin.com





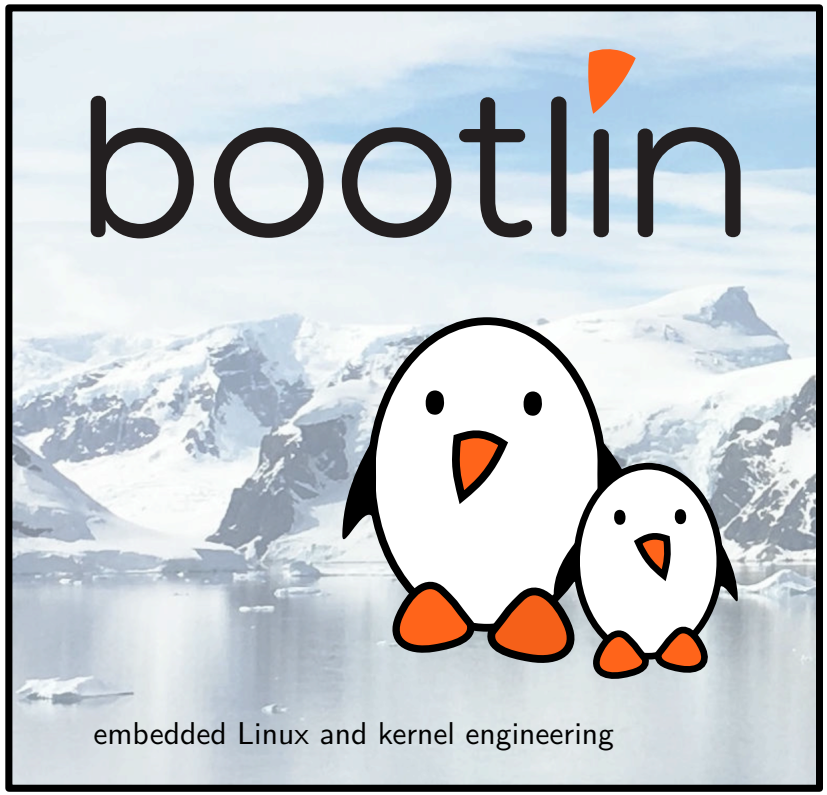
Yocto Project and OpenEmbedded system development training

- These slides are the training materials for Bootlin's *Yocto Project and OpenEmbedded system development* training course.
- If you are interested in following this course with an experienced Bootlin trainer, we offer:
  - **Public online sessions**, opened to individual registration. Dates announced on our site, registration directly online.
  - **Dedicated online sessions**, organized for a team of engineers from the same company at a date/time chosen by our customer.
  - **Dedicated on-site sessions**, organized for a team of engineers from the same company, we send a Bootlin trainer on-site to deliver the training.
- Details and registrations: https://bootlin.com/training/yocto
- Contact: `training@bootlin.com`



Icon by Eucalyp, Flaticon

About Bootlin



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Bootlin introduction

- Engineering company
  - In business since 2004
  - Before 2018: *Free Electrons*
- Team based in France and Italy
- Serving **customers worldwide**
- **Highly focused and recognized expertise**
  - Embedded Linux
  - Linux kernel
  - Embedded Linux build systems
- **Strong open-source** contributor
- Activities
  - **Engineering** services
  - **Training** courses
- https://bootlin.com

bootlin



Bootlin engineering services

Bootloader / firmware development

U-Boot, Barebox, OP-TEE, TF-A, .../

Linux kernel porting and driver development

Linux BSP development, maintenance and upgrade

Embedded Linux build systems

Yocto, OpenEmbedded, Buildroot, ...

Embedded Linux integration

Boot time, real-time, security, multimedia, networking

Open-source upstreaming

Get code integrated in upstream Linux, U-Boot, Yocto, Buildroot, ...



Bootlin training courses

Embedded Linux system development On-site: 4 or 5 days Online: 7 * 4 hours	Linux kernel driver development On-site: 5 days Online: 7 * 4 hours	Yocto Project system development On-site: 3 days Online: 4 * 4 hours	Buildroot system development On-site: 3 days Online: 5 * 4 hours	Embedded Linux networking On-site: 3 days Online: 4 * 4 hours
Understanding the Linux graphics stack On-site: 2 days Online: 4 * 4 hours	Embedded Linux audio On-site: 2 days Online: 4 * 4 hours	Embedded Linux Security On-site: 3 days Online: 4 * 4 hours	Linux debugging, tracing, profiling and performance analysis On-site: 3 days Online: 4 * 4 hours	Real-Time Linux with PREEMPT_RT On-site: 2 days Online: 3 * 4 hours

All our training materials are freely available
under a free documentation license (CC-BY-SA 3.0)
See https://bootlin.com/training/



Bootlin, an open-source contributor

- Strong contributor to the **Linux** kernel
  - In the top 30 of companies contributing to Linux worldwide
  - Contributions in most areas related to hardware support
  - Several engineers maintainers of subsystems/platforms
  - 9000 patches contributed
  - https://bootlin.com/community/contributions/kernel-contributions/
- Contributor to **Yocto Project**
  - Maintainer of the official documentation
  - Core participant to the QA effort
- Contributor to **Buildroot**
  - Co-maintainer
  - 6000 patches contributed
- Significant contributions to U-Boot, OP-TEE, Barebox, etc.
- Fully **open-source training materials**

Bootlin on-line resources

- Website with a technical blog:
  https://bootlin.com
- Engineering services:
  https://bootlin.com/engineering
- Training services:
  https://bootlin.com/training
- LinkedIn:
  https://www.linkedin.com/company/bootlin
- Elixir - browse Linux kernel sources on-line:
  https://elixir.bootlin.com



Icon by Freepik, Flaticon

Generic course information



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!

Beaglebone Black / Beaglebone black wireless shopping list

- BeagleBone Black or BeagleBone Black Wireless, from BeagleBoard.org
  - Texas Instruments AM335x (ARM Cortex-A8 CPU)
  - 512 MB of RAM
  - 4 GB of on-board eMMC storage
  - Plenty of peripherals and features
  - 2 x 46 pins headers, with access to many expansion buses (I2C, SPI, UART and more)
- MicroUSB cable
- USB Serial Cable - 3.3 V - Female ends (for serial console)[1]
- Nintendo Nunchuk with UEXT connector[2]
- Breadboard jumper wires - Male ends (to connect the Nunchuk)[3]
- MicroSD card









[1] https://www.olimex.com/Products/USB-Modules/Interfaces/USB-SERIAL-F

[2] https://www.olimex.com/Products/Modules/Sensors/MOD-WII/MOD-Wii-UEXT-NUNCHUCK/

[3] https://www.olimex.com/Products/Breadboarding/JUMPER-WIRES/JW-110x10/

STM32MP157 shopping list

- Discovery Kits from STMicroelectronics: STM32MP157A-DK1, STM32MP157D-DK1, STM32MP157C-DK2 or STM32MP157F-DK2[1]
  - STM32MP157 (Dual Cortex-A7 + Cortex-M4) CPU
  - 512 MB DDR3L RAM
  - Plenty of peripherals: GPIOs, SPI, Serial, USB, Ethernet...
- MicroUSB cable (to access serial console)
- USB-C to USB-A cable (to power the board)
- Nintendo Nunchuk with UEXT connector[2]
- Breadboard jumper wires - Male ends (to connect the Nunchuk)[3]
- MicroSD card
- RJ45 cable







[1] Boards documentation: A-DK1, D-DK1, C-DK2, F-DK2

[2] https://www.olimex.com/Products/Modules/Sensors/MOD-WII/MOD-Wii-UEXT-NUNCHUCK/

[3] https://www.olimex.com/Products/Breadboarding/JUMPER-WIRES/JW-110x10/

BeaglePlay shopping list

- BeaglePlay, from BeagleBoard.org
  - Texas Instruments AM625x (4×ARM Cortex-A53 CPU)
  - 2 GB RAM
  - 16 GB of on-board eMMC storage
  - Peripherals: SPI, I2C, UART, USB...
- USB-C cable for the power supply
- A USB-FTDI cable
- RJ45 cable for networking
- A micro SD card with at least 2G of capacity
- Nintendo Nunchuk with UEXT connector[1]
- Breadboard jumper wires - Male ends (to connect the Nunchuk)[2]







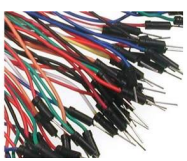
[1] https://www.olimex.com/Products/Modules/Sensors/MOD-WII/MOD-Wii-UEXT-NUNCHUCK/

[2] https://www.olimex.com/Products/Breadboarding/JUMPER-WIRES/JW-110x10/

IMX93 FRDM shopping list

- NXP i.MX93 11x11 FRDM board (76 EUR + VAT on Mouser)
  - NXP i.MX93 (Dual ARM Cortex-A55 + Cortex-M33)
  - 2 GB LPDDR4
  - 32 GB of on-board eMMC storage
  - Plenty of peripherals: I2C, SPI, UART, USB...
- 2 USB-C cable for the power supply and the serial console
- RJ45 cable for networking
- MicroSD card
- Nintendo Nunchuk with UEXT connector[1]
- Breadboard jumper wires - Male/Female ends (to connect to Nunchuk)[2]





[1] https://www.olimex.com/Products/Modules/Sensors/MOD-WII/MOD-Wii-UEXT-NUNCHUCK/

[2] https://www.olimex.com/Products/Breadboarding/JUMPER-WIRES/JW-200x10-FM/



Training quiz and certificate

- To get your training certificate you must
  1. Attend all sessions of this training course
  2. Achieve more than 50% of correct answers at our final quiz
     - The final quiz questions are identical to the pre-training quiz
     - The final quiz must be completed within two weeks of the session end's date
- The training certificate will be sent to you two weeks after the session end's date.



Participate!

During the lectures...

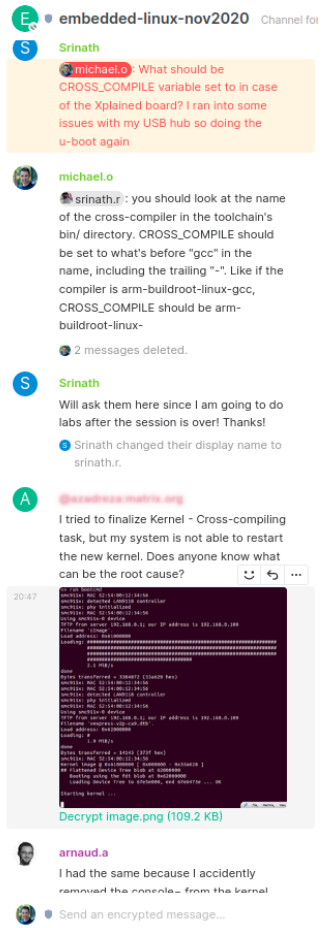
- Don't hesitate to ask questions. Other people in the audience may have similar questions too.
- Don't hesitate to share your experience too, for example to compare Linux with other operating systems you know.
- Your point of view is most valuable, because it can be similar to your colleagues' and different from the trainer's.
- In on-line sessions
  - Please always keep your camera on!
  - Also make sure your name is properly filled.
  - You can also use the "Raise your hand" button when you wish to ask a question but don't want to interrupt.
- All this helps the trainer to engage with participants, see when something needs clarifying and make the session more interactive, enjoyable and useful for everyone.



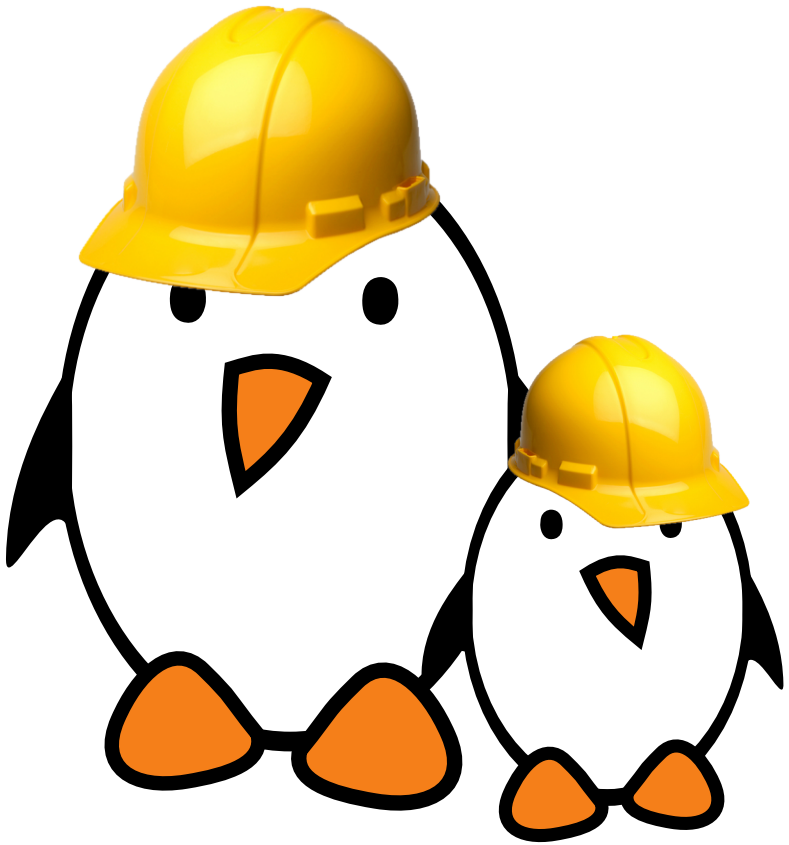
Collaborate!

As in the Free Software and Open Source community, collaboration between participants is valuable in this training session:

- Use the dedicated Matrix channel for this session to add questions.
- If your session offers practical labs, you can also report issues, share screenshots and command output there.
- Don't hesitate to share your own answers and to help others especially when the trainer is unavailable.
- The Matrix channel is also a good place to ask questions outside of training hours, and after the course is over.



Practical lab - Training Setup



Prepare your lab environment

- Download and extract the lab archive

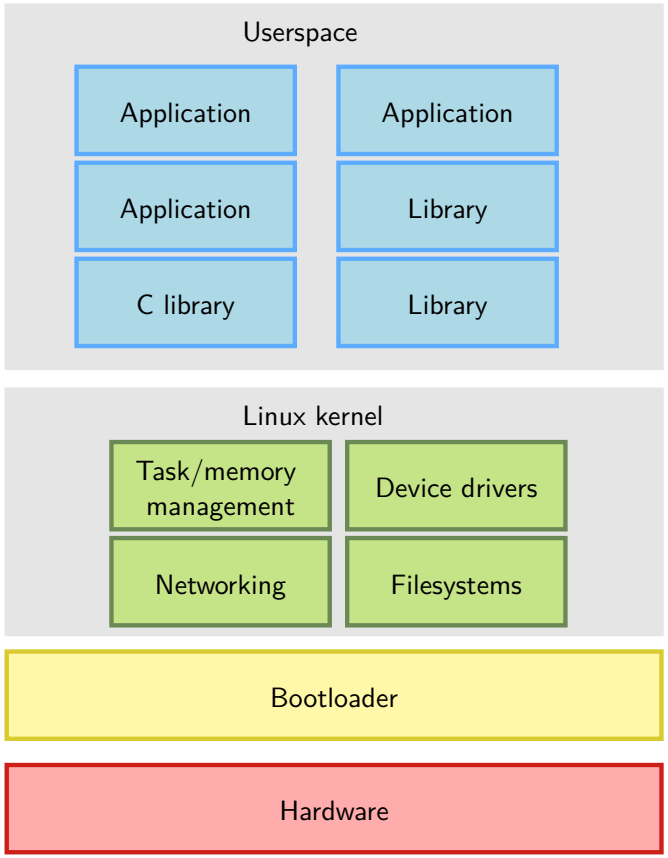
Introduction to Embedded Linux



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!

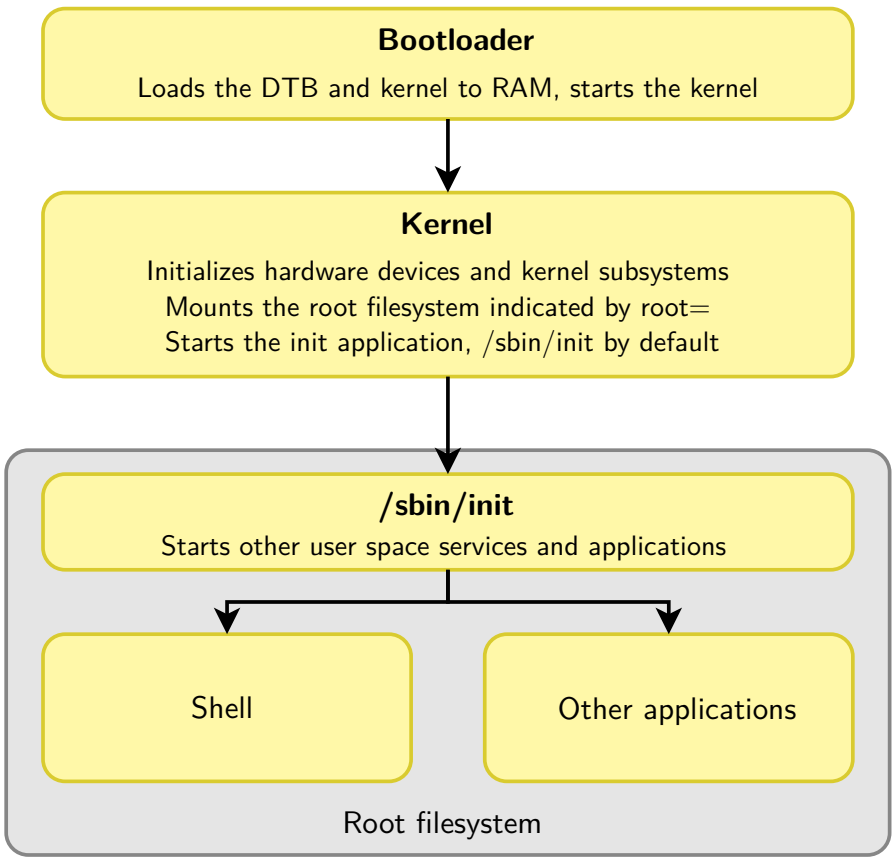


Simplified Linux system architecture





Overall Linux boot sequence

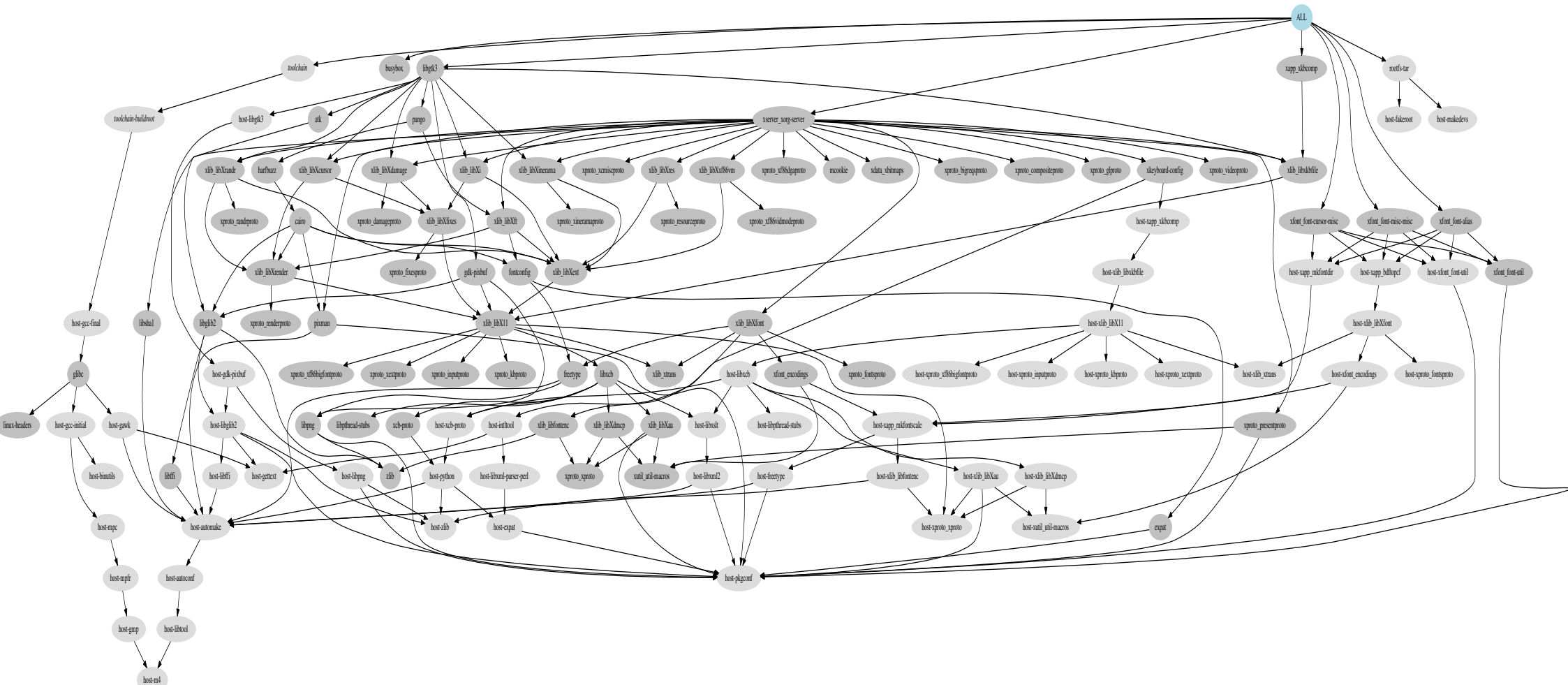




Embedded Linux work

- **BSP work**: porting the bootloader and Linux kernel, developing Linux device drivers.
- **system integration work**: assembling all the user space components needed for the system, configure them, develop the upgrade and recovery mechanisms, etc.
- **application development**: write the company-specific applications and libraries.

Complexity of user space integration

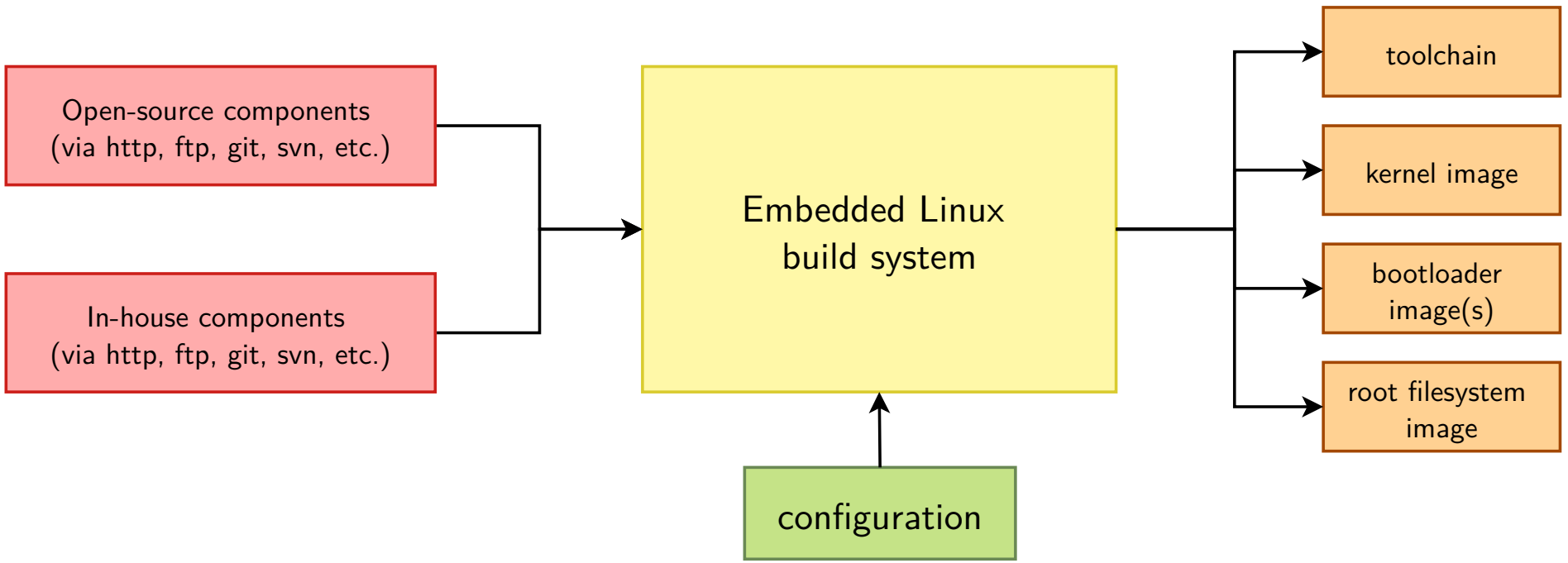




System integration: several possibilities

	Pros	**Cons**
Building everything manually	Full flexibility Learning experience	Dependency hell Need to understand a lot of details Version compatibility Lack of reproducibility
Binary distribution Debian, Ubuntu, Fedora, etc.	Easy to create and extend	Hard to customize Hard to optimize (boot time, size) Hard to rebuild the full system from source Large system Uses native compilation (slow) No well-defined mechanism to generate an image Lots of mandatory dependencies Not available for all architectures
Build systems Buildroot, Yocto, PTXdist, etc.	Nearly full flexibility Built from source: customization and optimization are easy Fully reproducible Uses cross-compilation Have embedded specific packages not necessarily in desktop distros Make more features optional	Not as easy as a binary distribution Build time

Embedded Linux build system: principle



- Building from source → lot of flexibility
- Cross-compilation → leveraging fast build machines
- Recipes for building components → easy



Embedded Linux build system: tools

- A wide range of solutions: Yocto/OpenEmbedded, PTXdist, Buildroot, OpenWRT, and more.
- Today, two solutions are emerging as the most popular ones
  - **Yocto/OpenEmbedded** Builds a complete Linux distribution with binary packages. Powerful, but somewhat complex, and quite steep learning curve.
  - **Buildroot** Builds a root filesystem image, no binary packages. Much simpler to use, understand and modify.

Yocto Project and core layers overview



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



The Yocto Project overview

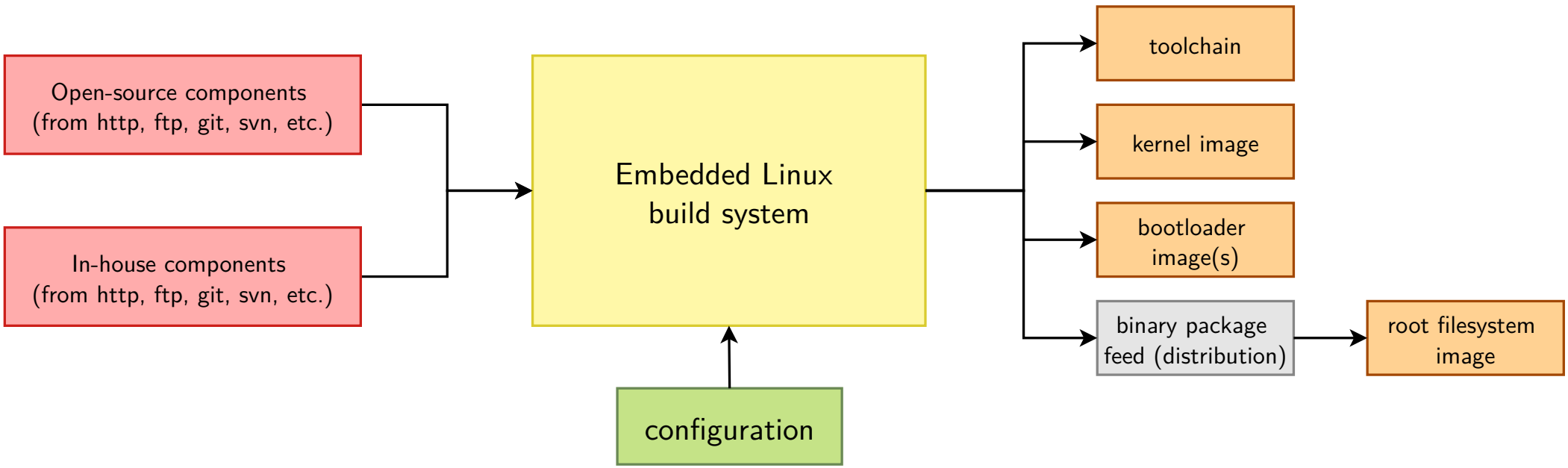


About

- The Yocto Project is an open source collaboration project that allows to build custom embedded Linux-based systems.
- Established by the Linux Foundation in 2010 and still managed by one of its fellows: Richard Purdie.



Yocto: principle



- Yocto always builds binary packages (a “distribution”)
- The final root filesystem is generated from the package feed
- The big picture is way more complex



Lexicon: `bitbake`

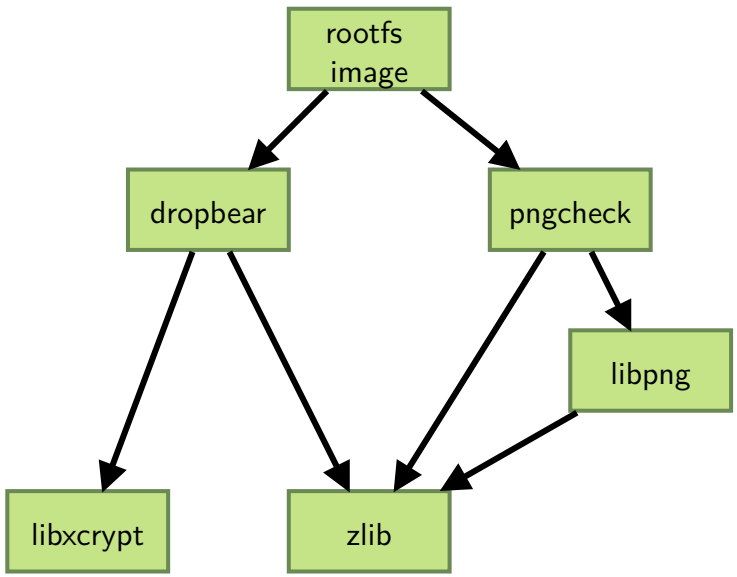
In Yocto / OpenEmbedded, the *build engine* is implemented by the `bitbake` program

- `bitbake` is a task scheduler, like `make`
- `bitbake` parses text files to know what it has to build and how
- It is written in Python (need Python 3 on the development host)



Lexicon: recipes

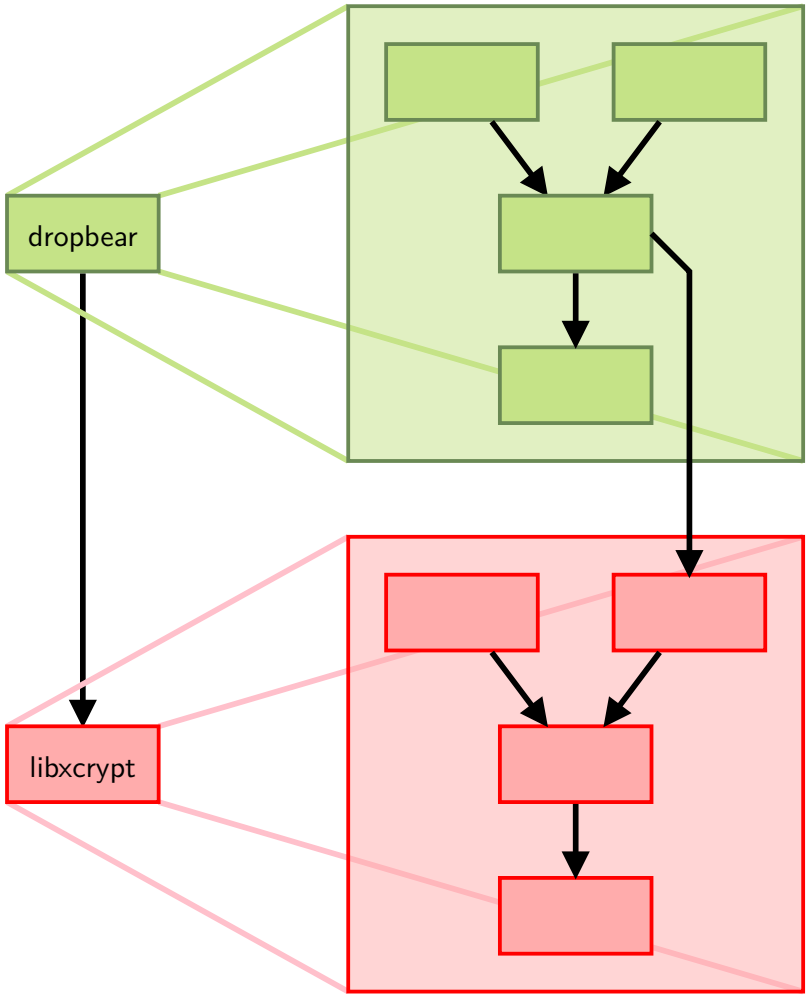
- The main kind of text file parsed by `bitbake` is *recipes*, each describing a specific software component
- Each *Recipe* describes how to fetch and build a software component: e.g. a program, a library or an image
- They have a specific syntax
- `bitbake` can be asked to build any recipe, building all its dependencies automatically beforehand





Lexicon: tasks

- The build process implemented by a recipe is split in several *tasks*
- Each task performs a specific step in the build
- Examples: fetch, configure, compile, package
- Tasks can depend on other tasks (including on tasks of other recipes)





Lexicon: metadata and layers

- The input to `bitbake` is collectively called *metadata*
- Metadata includes *configuration files*, *recipes*, *classes* and *include files*
- Metadata is organized in *layers*, which can be composed to get various components
  - A layer is a set of recipes, configurations files and classes matching a common purpose
    - For Texas Instruments board support, the *meta-ti-bsp* layer is used
  - Multiple layers are used for a project, depending on the needs
- *openembedded-core* is the core layer
  - All other layers are built on top of openembedded-core
  - It supports the ARM, MIPS (32 and 64 bits), PowerPC, RISC-V and x86 (32 and 64 bits) architectures
  - It supports QEMU emulated machines for these architectures

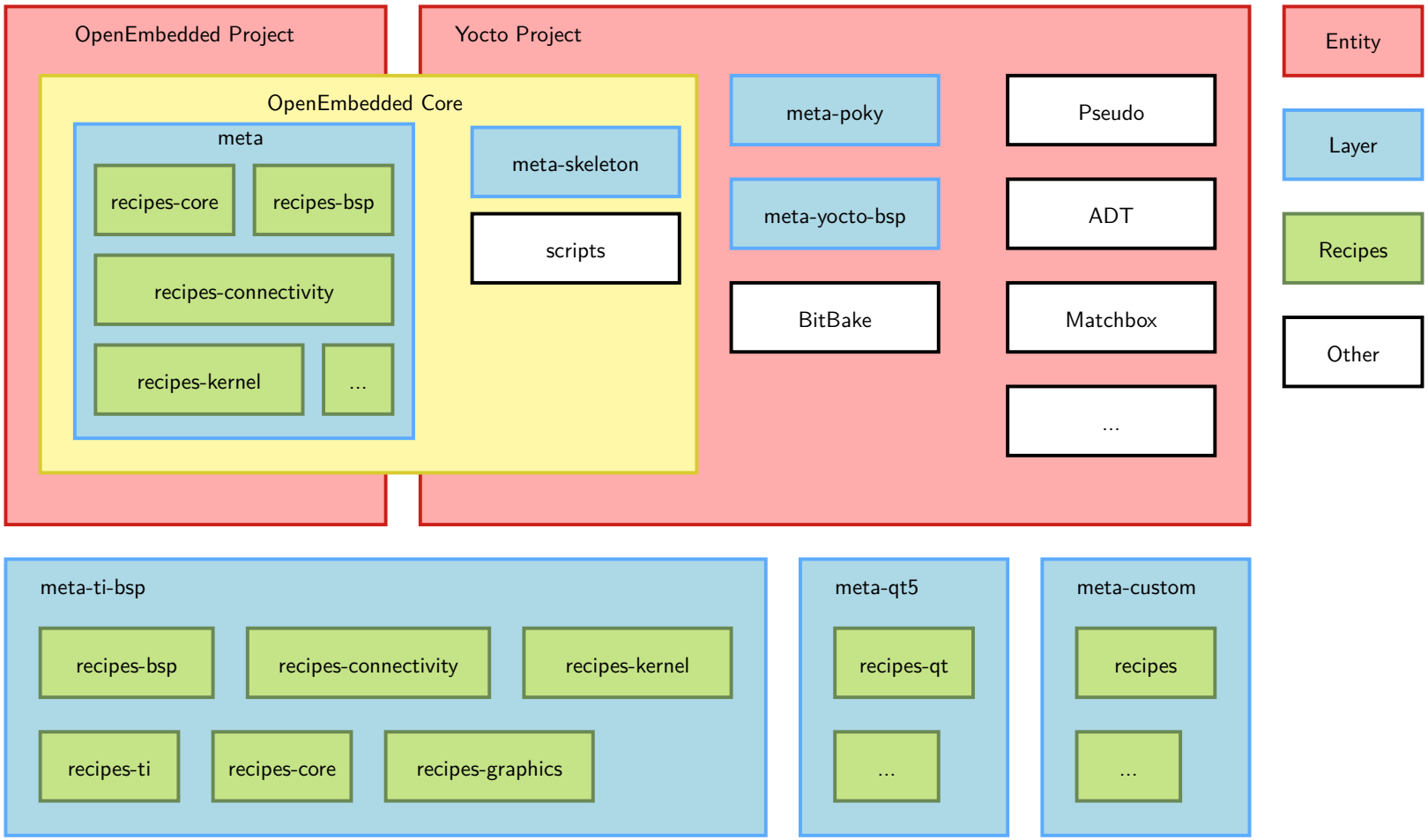


Lexicon: Poky

- The word *Poky* has several meanings
- poky is the *reference distro* provided by the Yocto Project
- meta-poky is the layer providing the poky reference distribution
- Up to the Walnascar (5.2) release, Poky was a git repository that was assembled from other git repositories: bitbake, openembedded-core, yocto-docs and meta-yocto



The Yocto Project lexicon





The Yocto Project lexicon

- The Yocto Project is **not used as** a finite set of layers and tools.
- Instead, it provides a **common base** of tools and layers on top of which custom and specific layers are added, depending on your target.



Example of a Yocto Project based BSP

- To build images for a BeagleBone Black, we need:
  - The Yocto core layers, containing all common recipes and tools.
  - The *meta-ti-bsp* layer, a set of Texas Instruments specific recipes.
- All modifications are made in your own layer. Editing openembedded-core or any other third-party layer is a **no-go**!
- We will set up this environment in the lab.



The Yocto core layers overview



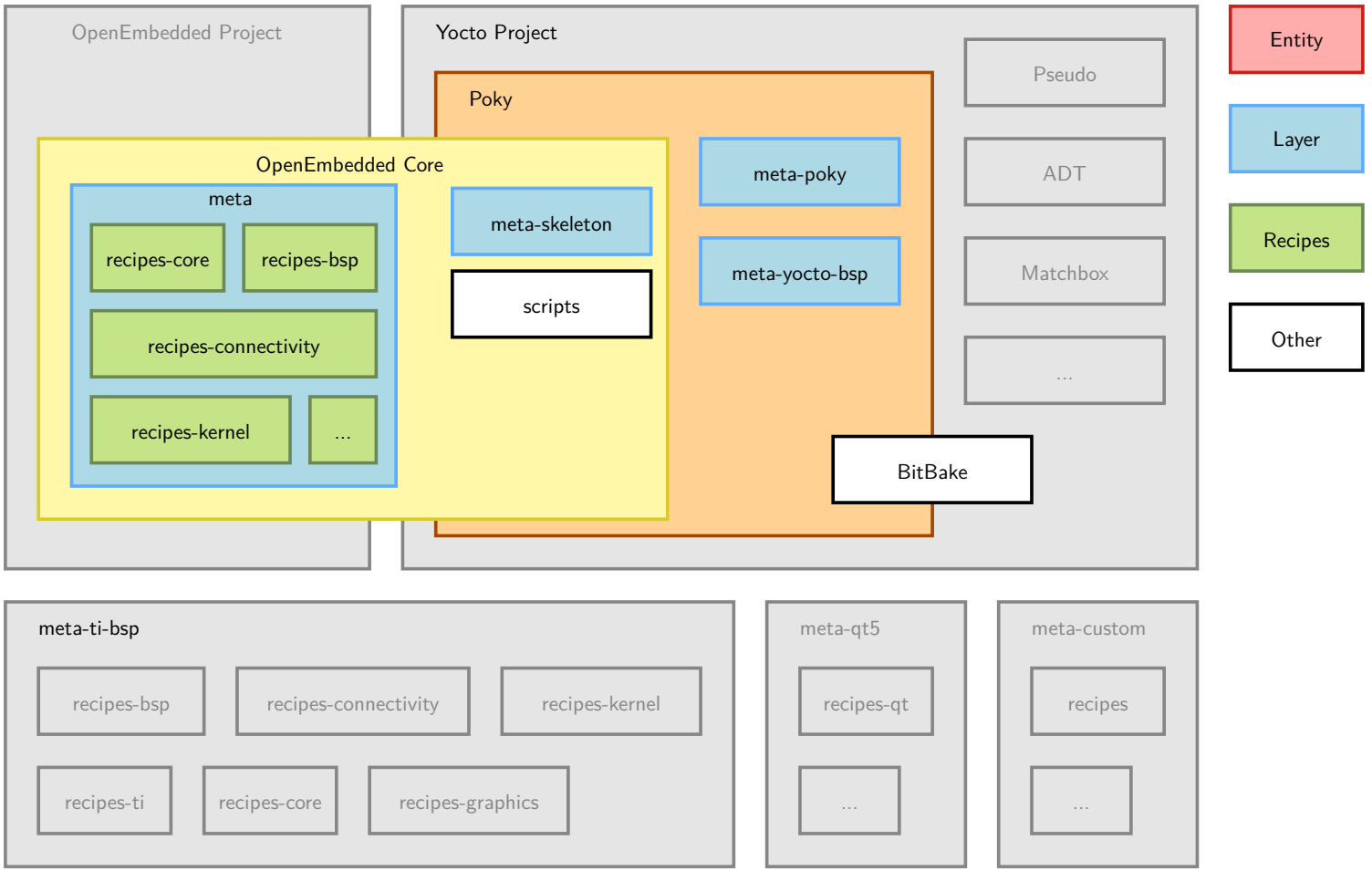
Getting the Poky reference system (up to Walnascar)

All official projects part of the Yocto Project are available at https://git.openembedded.org/ and https://git.yoctoproject.org/

- To download the Poky reference system:
  `git clone -b scarthgap https://git.yoctoproject.org/git/poky`
- A new version is released every 6 months, and maintained for 7 months
- **LTS** versions are maintained for 4 years, and announced before their release.
- Each release has a codename such as `kirkstone` or `scarthgap`, corresponding to a release number.
  - A summary can be found at https://www.yoctoproject.org/development/releases/



Poky


OpenEmbedded Project
Yocto Project
Poky
OpenEmbedded Core
meta
recipes-core
recipes-bsp
recipes-connectivity
recipes-kernel
...
meta-skeleton
scripts
meta-poky
meta-yocto-bsp
BitBake
Pseudo
ADT
Matchbox
...
Entity
Layer
Recipes
Other
meta-ti-bsp
recipes-bsp
recipes-connectivity
recipes-kernel
recipes-ti
recipes-core
recipes-graphics
meta-qt5
recipes-qt
...
meta-custom
recipes
...



Poky source tree

bitbake/	Holds all scripts used by the `bitbake` command. Usually matches the stable release of the BitBake project.
documentation/	All documentation sources for the Yocto Project documentation. Can be used to generate nice PDFs.
meta/	Contains the OpenEmbedded-Core metadata.
meta-skeleton/	Contains template recipes for BSP and kernel development.
meta-poky/	Holds the configuration for the Poky reference distribution.
meta-yocto-bsp/	Configuration for the Yocto Project reference hardware board support package.
LICENSE	The license under which Poky is distributed (a mix of GPLv2 and MIT).
oe-init-build-env	Script to set up the OpenEmbedded build environment. It will create the build directory.
scripts/	Contains scripts used to set up the environment, development tools, and tools to flash the generated images on the target.



Getting the Yocto reference layers (starting from Whinlatter)

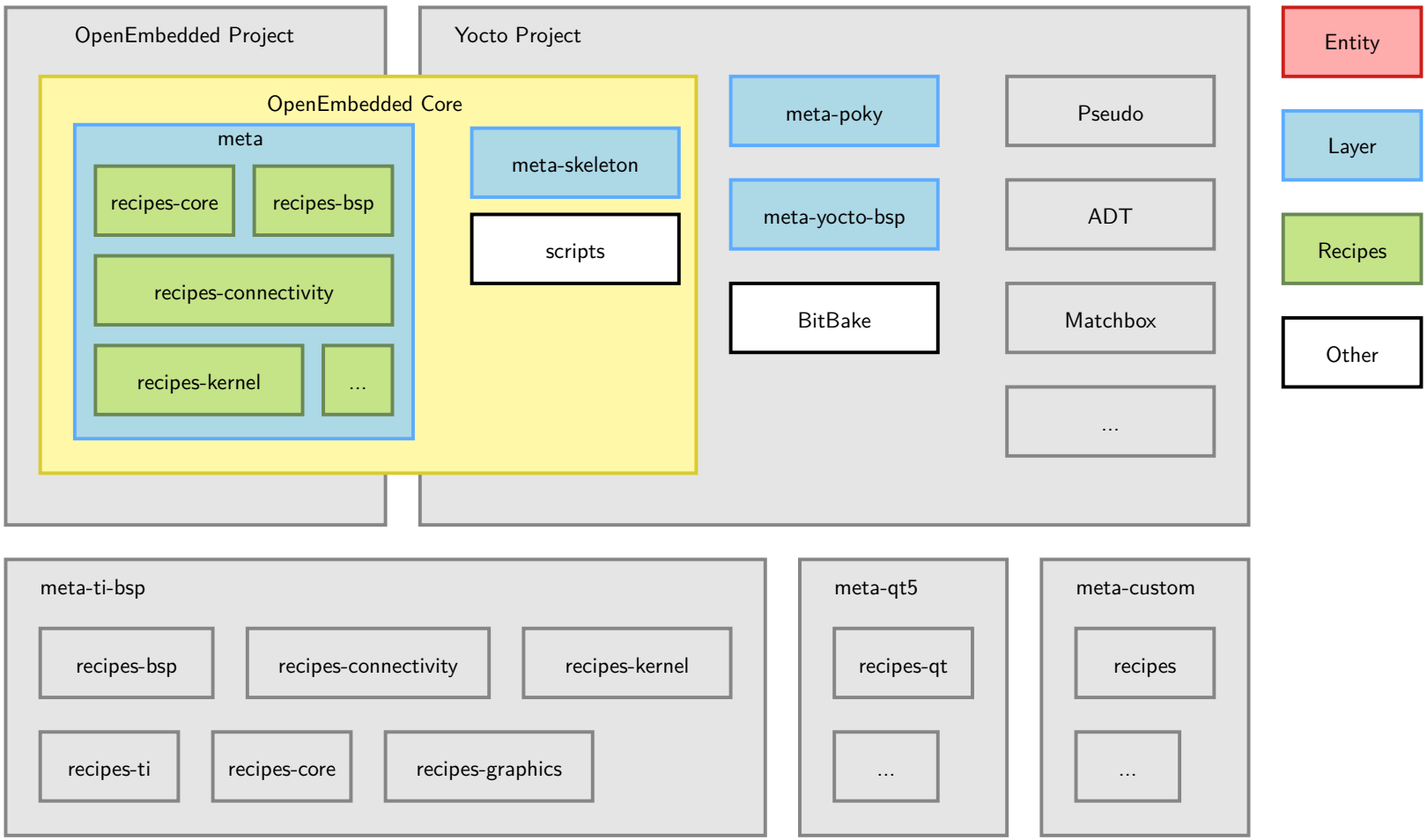
- The assembled poky git repository was deprecated with the Whinlatter (5.3) release.
- All source code is still the same but needs to be assembled from separate gits:
  - Either manually:

```
git clone https://git.openembedded.org/bitbake -b yocto-6.0
git clone https://git.openembedded.org/openembedded-core -b yocto-6.0
git clone https://git.yoctoproject.org/meta-yocto -b yocto-6.0
```

  - Or using the new bitbake-setup utility.
- Everything else stays the same.

```
poky/bitbake/                  bitbake/
poky/meta/                     openembedded-core/meta/
poky/oe-init-build-env    →    openembedded-core/oe-init-build-env
poky/meta-poky/                meta-yocto/meta-poky/
poky/meta-yocto-bsp/           meta-yocto/meta-yocto-bsp/
```

Getting the Yocto reference layers (starting from Whinlatter)





Yocto core layers source tree

bitbake/	Holds all scripts used by the `bitbake` command. Usually matches the stable release of the BitBake project.
openembedded-core/meta/	Contains the OpenEmbedded-Core metadata.
openembedded-core/meta-skeleton/	Contains template recipes for BSP and kernel development.
openembedded-core/LICENSE	The license under which openembedded-core is distributed (a mix of GPLv2 and MIT).
openembedded-core/oe-init-build-env	Script to set up the OpenEmbedded build environment. It will create the build directory.
openembedded-core/scripts/	Contains scripts used to set up the environment, development tools, and tools to flash the generated images on the target.
meta-yocto/meta-poky/	Holds the configuration for the Poky reference distribution.
meta-yocto/meta-yocto-bsp/	Configuration for the Yocto Project reference hardware board support package.



Documentation

- Documentation for the current sources, compiled as a “mega manual”, is available at: https://docs.yoctoproject.org/singleindex.html
- Variables in particular are described in the variable glossary: https://docs.yoctoproject.org/genindex.html